

A Zero-Training Error Correction System with Large Language Models

Yangyang Wu^{*1}, Chen Yang^{*2}, Mengying Zhu^{*3}, Xiaoye Miao^{†#4}, Wei Ni^{†*5},
Meng Xi^{*6}, Xinkui Zhao^{*7}, Jianwei Yin^{‡8}

^{*}Software College, Zhejiang University, Ningbo, China

[†]Center for Data Science, Zhejiang University, Hangzhou, China

[#]The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China

^{*}Department of Data Science, City University of Hong Kong, Hong Kong, China

[‡]College of Computer Science, Zhejiang University, Hangzhou, China

{¹zjuwuuy, ²zjuyc, ³mengyingzhu, ⁴miaoxy, ⁵wei.ni, ⁶ximeng, ⁷zhaoxinkui}@zju.edu.cn, ⁸zjuyjw@cs.zju.edu.cn

Abstract—Correcting missing or erroneous data values is an essential task in data cleaning. Traditional pre-configuration error correction (EC) methods rely heavily on predefined rules or constraints, demanding significant domain knowledge and manual effort. While configuration-free EC approaches have been explored, they still demand extensive feature engineering or labeled data for intensive model training. In this paper, we propose a zero-training and interpretable EC system, named **ZeroEC**, that leverages large language models (LLMs) to generate chain-of-thoughts (CoTs) and correction rules for EC, without the need for model training. **ZeroEC** consists of two modules, *contextual-relevant tuple search* (CTS) and *training-free explainable correction* (TEC). CTS constructs a *contextual-relevant tuple retriever* using a *weighted cosine similarity* function to efficiently identify the most relevant tuples for each dirty tuple, reducing redundancy in the LLM prompts and lowering computational costs. TEC employs a *clustering-based representative tuple sampling* strategy to alleviate “hallucination” risk by exposing LLMs to diverse types of data errors. It further prompts for generating *correction CoTs* for user-corrected representative tuples, as well as prompts for creating *correction rules* and *explainable ECs*, which automatically provide explanations for EC, all without the need for model training. Extensive experiments conducted on various real-world datasets demonstrate that **ZeroEC** achieves a 66.82% increase in accuracy and a 6.87x speedup compared to state-of-the-art methods. The codes and datasets of this paper are available at <https://github.com/YangChen32768/ZeroEC>.

Index Terms—Error Correction, Correction Rule Generation, Correction Chain-of-thoughts, Large Language Models

I. INTRODUCTION

Data cleaning is a fundamental yet challenging step in various relational data engineering tasks [1], [2], such as entity matching, data integration, or analytics. It typically involves two stages: *error detection* (ED) to identify incorrect values, and *error correction* (EC) to fix them [3]–[6]. While ED studies [5], [7]–[9] have achieved obvious advancements, achieving an average F1 score above 0.85 on real datasets [5], [7], the EC task remains more challenging. Despite employing techniques like machine learning [4], [10], [11], transfer learning [12], and boosting [13], current EC methods often fall short, with average F1 scores below 0.7 in real-world scenarios [4], [11], [14]. Therefore, exploring more effective EC systems for real-life applications remains a crucial need.

Traditional EC systems follow the pre-configuration paradigm [11], [15], where users have to define different types of rules or constraints [3], [11], [15], [16], such as *functional dependencies* (FDs) [17] and *denial constraints* (DCs) [11]. However, this requirement demands expert knowledge of both datasets and the EC system [3], [18], a level of expertise that most users do not possess, thereby limiting its practicality for non-expert users in real-world scenarios. To address this issue, several configuration-free EC approaches [4], [10] have been proposed, employing pruning strategies or feature engineering alongside machine learning models [4]. While these approaches reduce the dependency on expert knowledge, they still require extensive feature engineering or labeled data for intensive model training, leading to inefficiencies and challenges in handling complex scenarios, such as undefined error patterns and intricate textual errors [18], [19].

Recently, large language models (LLMs) have shown promise in solving data cleaning (DC) tasks by transforming them into generation tasks, such as error detection [20]–[24] and data imputation [20]–[22], [25], [26].

With billions of parameters pre-trained on massive text data, LLMs exhibit remarkable capabilities in adapting to new DC tasks through few-shot generalization [27], [28]. Their effectiveness is primarily attributed to two key factors: (i) few-shot prompting, which enables LLMs to handle novel generation tasks with only a few examples; and (ii) the extensive prior knowledge embedded in their parameters.

Directly applying LLMs to the EC task without a well-designed framework presents several critical challenges: *First*, due to token limitations, LLMs struggle to capture necessary contexts from massive relational data, resulting in inefficiencies and unreliable corrections (**CH1**). *Second*, unique and non-standard data errors can induce LLMs to generate plausible but incorrect or fabricated corrections, a phenomenon known as “hallucination” [29], [30] (**CH2**). *Third*, LLM-generated corrections often lack the interpretability required by mission-critical data systems such as healthcare, finance, and government (**CH3**). These limitations underscore the necessity of developing a systematic EC framework that not only integrates LLMs effectively but also addresses the inherent

challenges posed by existing EC solutions.

To address the above challenges, in this paper, we propose a *zero-training and interpretable EC* system **ZeroEC**, that leverages LLMs to utilize their implicit prior knowledge for efficiently and effectively correcting erroneous data and generating reasoning chain-of-thoughts (CoTs), without model training. It follows the configuration-free paradigm, where user supervision is exercised only in the form of annotating a few data values, a task that is less cognitively demanding than composing rules or constraints (in Sect. II). **ZeroEC** is composed of two modules, a *contextual-relevant tuple search* (CTS) module and a *training-free explainable correction* (TEC) module. CTS constructs a *customized contextual-relevant tuple* (CT) retriever with a *weighted cosine similarity* (WS) function to find the most contextually correlated tuples for each dirty tuple, instead of using the entire dataset (*for addressing CH1*). TEC puts forward a *clustering-based representative tuple sampling* (CS) strategy to expose LLMs to a wide variety of data error types and their corresponding manual corrections, thereby reducing the risk of “hallucination” during LLM-generated correction (*for addressing CH2*). In addition, TEC designs prompts for *correction CoT generation*, *EC rule generation*, and *explainable EC*, ensuring interpretable corrections for critical applications (*for addressing CH3*). In summary, the main contributions are as follows.

- (1) *Zero-training System*. We propose **ZeroEC**, a novel zero-training, interpretable EC system that leverages LLMs to correct erroneous data and generate reasoning chain-of-thoughts (CoTs) efficiently and effectively, without requiring model training (Sect. III).
- (2) *Contextual-relevant Tuple Search*. The CTS module (in Sect. IV) introduces a CT retriever that utilizes the WS function to efficiently identify the most contextually relevant tuples for each dirty tuple. This significantly shortens the prompts of LLMs and lowers computational costs.
- (3) *Explainable Correction*. The TEC module (in Sect. V) employs the CS strategy with a *correction CoT generation* prompt, leveraging few-shot prompting to automatically generate *correction CoTs* for user-corrected representative tuples. In addition, it incorporates prompts for *correction rules generation* and *explainable EC*, ensuring interpretable corrections through generated rules and CoTs.
- (4) *Extensive Experiments*. Extensive experiments (in Sect. VI) demonstrate the computation benefits and accuracy gains of **ZeroEC** over state-of-the-art methods.

II. PROBLEM DEFINITION

Let $\mathcal{D} = \{\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_{|\mathcal{D}|}\}$ be a relational dataset of size $|\mathcal{D}|$ with dirty data, where each $\mathbf{t}_i$ denotes a tuple. $\mathcal{A} = \{\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_{|\mathcal{A}|}\}$ is the schema of $\mathcal{D}$, with $|\mathcal{A}|$ attributes. We define t_{ij} as the data value in tuple $\mathbf{t}_i$ and attribute $\mathbf{a}_j$. Let $\mathcal{D}^*$ be the ground truth of $\mathcal{D}$ with only clean values. To encode the error distribution information of $\mathcal{D}$, a detection matrix $\mathbf{D}$ is defined as $\mathbf{D} = (\mathbf{d}_1, \dots, \mathbf{d}_{|\mathcal{D}|})^\top$. Each detection vector $\mathbf{d}_i = (d_{i1}, \dots, d_{i|\mathcal{A}|})$ corresponds to a tuple $\mathbf{t}_i$. d_{ij} takes value from $\{0, 1\}$, $i = \{1, \dots, |\mathcal{D}|\}$, and $j = \{1, \dots, |\mathcal{A}|\}$; $d_{ij} = 1$

TABLE I: Symbols and description

Symbol	Description
$\mathcal{D}$	an input dirty relational dataset (stored in a matrix)
$\mathcal{A}$	the schema of $\mathcal{D}$
$\mathbf{D}$	a detection matrix of $\mathcal{D}$
$\hat{\mathcal{D}}$	dataset corrected by the EC model
$\mathcal{D}^*$	the ground truth dataset w.r.t. $\mathcal{D}$
$\mathbf{t}_i$	the i -th tuple in $\mathcal{D}$
θ	the representative tuple size for ZeroEC
$\mathcal{M}$	error correction model

(resp. 0) *iff* the j -th dimension is clean (resp. error). Table I lists the frequently used symbols throughout this paper.

Aligned with prior studies [4], [31], we focus on correcting both *syntactic* and *semantic* errors in the error correction task. These errors encompass *missing values*, *typographical errors*, *formatting issues*, and *violated attribute dependencies*. In general, we formalize the *error correction* (EC) problem over the dirty relational dataset $\mathcal{D}$ with the detection matrix $\mathbf{D}$ in Definition 1.

Definition 1: (Error correction, EC). Given a dirty dataset $\mathcal{D}$ with the detection matrix $\mathbf{D}$, the EC problem aims to build one correction model $\mathcal{M}_j$ for each dirty column $\mathbf{a}_j$ in $\mathcal{D}$ to find appropriate values for dirty components in $\mathbf{a}_j$, i.e., the $\hat{t}_{ij}$ value in the *corrected* dataset $\hat{\mathcal{D}}$ can be formalized as: $\hat{t}_{ij} = \bar{t}_{ij} \cdot (1 - d_{ij}) + t_{ij} \cdot d_{ij}$, where $\bar{t}_{ij} = \mathcal{M}_j(\mathbf{t}_i)$ is the value predicted by $\mathcal{M}_j$ for $\mathbf{t}_i$. EC models aim to make the *corrected* dataset $\hat{\mathcal{D}}$ as *close* to the *real clean* dataset $\mathcal{D}^*$ as possible.

Example 1: Table II shows a *hospital* dataset with 6 columns and 8 tuples. Errors are highlighted in “red” and corrections in “blue”. The EC system aims to fix these errors for improved accuracy and cleanliness. For instance, in tuple $\mathbf{t}_1$, a typographical error in the “Drug” field is corrected from “Digoxxn” to “Digoxin”. In tuple $\mathbf{t}_2$, a missing value in the “State” field is filled from “NULL” to “AL”. In tuple $\mathbf{t}_3$, the value “CA” violates the attribute dependency City $\rightarrow$ State, necessitating correction to “AL”. In tuple $\mathbf{t}_4$, the value “65” contradicts the attribute dependency that “Drug Dosage” should be calculated as 80% of “Weight”, leading to the correction “52”. In tuple $\mathbf{t}_7$, “hx-2” exhibits both a typographical error and a format inconsistency, as “Stateavg” should combine “State” and “Measure” with an underscore, making the correct value “fl_hf-2”.

III. SYSTEM OVERVIEW

In this section, we present an overview of the proposed **ZeroEC** system for *error correction* (EC) in dirty relational datasets. As analyzed previously, directly deploying LLMs for EC faces challenges related to token limitations and the handling of non-standard data errors, which can lead to inefficiencies, unreliable corrections, and “hallucination” phenomena. To address these issues, we propose **ZeroEC**, an efficient and effective EC system designed to leverage LLMs for correcting erroneous data values.

Figure 1 illustrates the framework of **ZeroEC**, which comprises two main modules: *contextual-relevant tuples search*

TABLE II: EC illustration using a *hospital* dataset

ID	City	State	Measure	Drug	Stateavg	Weight	Drug Dosage
t_1	Boaz	AL	pn-3b	Digoxin → Digoxin	pn-3b → al_pn-3b	75	75 → 60
t_2	Boaz	NULL → AL	ami-1	Digoxin	al_ami-1	78	62.4
t_3	Dothan	CA → AL	ami-2	Digoxin	ami-2 → al_ami-2	79	63.2
t_4	Dothan	AL	ami-2	Digoxin	al_ami-2	65	65 → 52
t_5	chicago → Chicago	IL	hf-1	Digoxin	il_hf-1	75	60
t_6	Chicago	IL	hf-1	Digoxin	il_hf-1	75	60
t_7	Mxami → Miami	FL	hf-2	Digoxin	hx-2 → fl_hf-2	78	78 → 62.4
t_8	Miami	FL	hf-2	Digoxin	fl_hf-2	78	62.4

(CTS) and *training-free explainable correction* (TEC). Its workflow consists of five consecutive steps: ① *contextual-relevant tuple search* in the CTS module, followed by ② *representative tuple sampling*, ③ *correction chain-of-thought (CoT) generation*, ④ *correction rule generation*, and ⑤ *explainable error correction*, all within the TEC module. ZeroEC takes a dirty relational dataset $\mathcal{D}$ as input and outputs the correction results, along with explanatory materials including validated correction rules for certain dirty columns and the corresponding correction CoTs for all dirty tuples.

Step ①: Contextual-relevant Tuples Search. Given the dirty relational dataset $\mathcal{D}$ and its dirty tuples $\mathcal{D}_{\text{err}}$ (in Step ①), the CTS module constructs a contextual-relevant tuple (CT) retriever $\mathcal{R}$ for each dirty column $N \in \mathcal{D}$. This retriever identifies contextually correlated tuples that serve as complementary information for correction, eliminating the need to include the entire dataset in prompt construction. By efficiently selecting the most relevant tuples for each dirty tuple, the CT retriever significantly shortens LLM prompts and reduces computational costs during EC. Specifically, the CT retriever $\mathcal{R}$ first leverages a *mutual information based column selection* strategy to select columns highly correlated with the dirty column N , for efficient data retrieval. It then employs the approximate nearest neighbor search library Meta Faiss [32] with the *weighted cosine similarity* $\text{Sim}(\mathbf{t}_i, \mathbf{t}_j)$ to identify the top $p(\ll |\mathcal{D}|)$ most relevant tuples $\mathcal{D}_p$ for each dirty tuple. Finally, the CT retriever constructs the most relevant tuple set by re-ranking the top $k(\ll p)$ tuples from $\mathcal{D}_p$ based on two criteria: (i) whether the target column is clean, and (ii) prioritizing tuples with more clean values among those sharing the same detection status of the target column.

Step ②: Clustering-based Representative Tuples Sampling. With the support of the CT retriever $\mathcal{R}$, the TEC module selects representative tuples for user corrections, and finds the most contextual-relevant tuples for them. This sampling exposes LLMs to various data error types, thereby reducing the risk of “hallucination” during LLM-generated ECs. Specifically, it first develops a *clustering-based representative tuple sampling* strategy, using the K -means clustering algorithm to divide all dirty tuples in the dirty tuple set $\mathcal{D}_{\text{err}}$ into θ clusters. It then introduces a greedy selection algorithm to select one representative dirty tuple from each cluster for user labeling, forming the representative tuple set $\mathcal{D}_{\text{err}}^{\text{rep}}$, its user-corrected

version $\hat{\mathcal{D}}_{\text{err}}^{\text{rep}}$, and the to-be-corrected dirty tuple set $\mathcal{D}_{\text{err}}^{\text{rem}}$.

Step ③: Correction CoT Generation. With the contextual relevant tuples retrieved for $\mathcal{D}_{\text{err}}^{\text{rep}}$, the TEC module designs an *automatic correction CoT generation* (CG) prompt using few-shot prompting to automatically generate CoTs for the EC results of representative tuples corrected by the user. The CG prompt takes task description, general CoT examples, representative tuples $\mathcal{D}_{\text{err}}^{\text{rep}}$, retrieved tuples $\mathcal{R}(\mathcal{D}_{\text{err}}^{\text{rep}})$, and user-corrected representative tuples $\hat{\mathcal{D}}_{\text{err}}^{\text{rep}}$ as inputs. It then automatically produces reasoning CoTs for corrections over $\mathcal{D}_{\text{err}}^{\text{rem}}$.

Step ④: Correction Rule Generation. Using the user-corrected representative tuples $\hat{\mathcal{D}}_{\text{err}}^{\text{rep}}$ and their corresponding generated correction CoTs, the TEC module develops a *high-quality correction rule generation* (RG) strategy. This strategy integrates an *EC rule generation* (RG) prompt and a *generated rule validation* (RV) mechanism to generate reliable EC rules, ensuring explainability and efficiency while reducing costly LLM invocations. Specifically, for each dirty column N , the prompt rephraser¹ generates n variations of the EC rule generation task description for each rule type φ , enhancing the diversity of EC rule candidates. Subsequently, the RG prompt guides LLMs to generate the EC rule candidate δ_i^φ for each φ . The input to this process includes the RG task description and the representative tuple correction workflow, which comprises the representative tuple, original dirty value, retrieved tuples, EC result, and reasoning correction CoT. The outputs is an EC rule candidate δ_i^φ . Finally, the validated EC rule set Σ_N for column N is constructed. This set includes only the generated rules δ_i^φ that successfully correct all dirty values in N of representative tuples $\mathcal{D}_{\text{err}}^{\text{rep}}$ while preserving the integrity of clean values in N of $\mathcal{D}_{\text{clean}}$.

Step ⑤: Explainable Error Correction. With the full EC processes of user-corrected representative tuples and generated high-quality EC rules, the TEC module designs a *hybrid explainable error correction* (HC) strategy to produce explainable EC results for each dirty value. Specifically, the HC strategy first applies all rules in Σ_N in parallel to derive multiple rule-based corrections and employs a majority voting mechanism to determine the final EC results, forming a rule-corrected set $\hat{\mathcal{D}}_{\text{err}}^{\text{rem}}$ for $\mathcal{D}_{\text{err}}^{\text{rem}}$. However, certain errors in $\hat{\mathcal{D}}_{\text{err}}^{\text{rem}}$ cannot be addressed solely by the *parallel rule execution* (RE) mechanism, as they involve scenarios where rules are either

¹ <https://github.com/f/awesome-chatgpt-prompts>

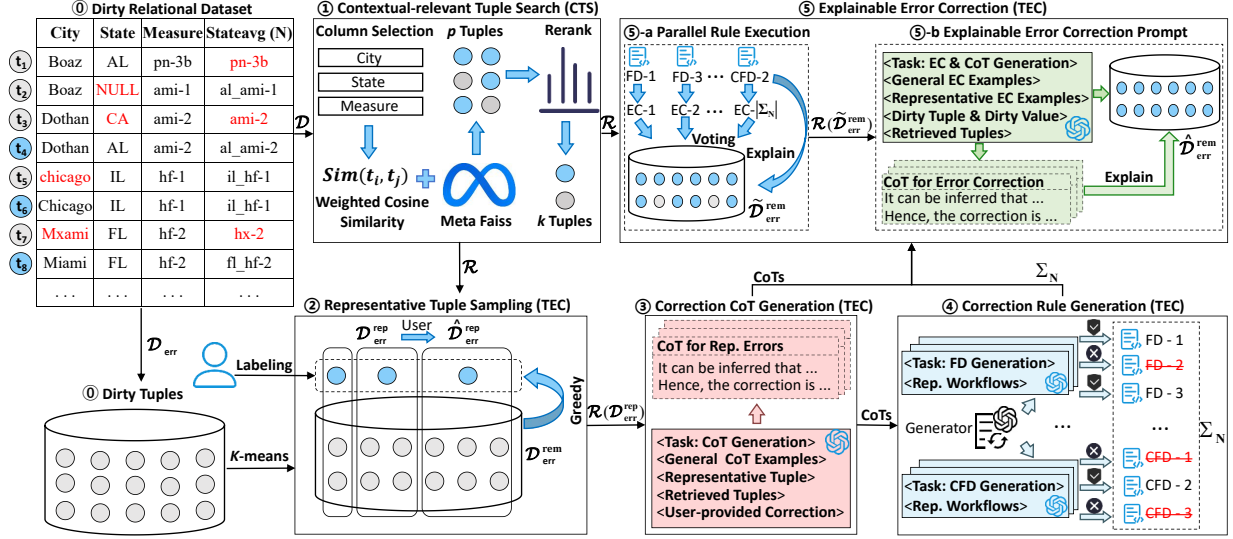


Fig. 1: The architecture of ZeroEC

unavailable, ineffective, or yield inconclusive results. Then, for such dirty values in $\tilde{\mathcal{D}}_{\text{err}}^{\text{rem}}$, the HC strategy introduces an *explainable error correction* prompt, which enables LLMs to generate both EC results and correction CoTs. It takes task description, general EC examples, representative examples generated by the CG prompt, dirty tuples in $\tilde{\mathcal{D}}_{\text{err}}^{\text{rem}}$, and retrieved tuples in $\mathcal{R}(\tilde{\mathcal{D}}_{\text{err}}^{\text{rem}})$ as inputs, outputting EC results for dirty tuples in $\tilde{\mathcal{D}}_{\text{err}}^{\text{rem}}$ and the corresponding reasoning CoTs.

IV. CTS MODULE

Feeding the entire relational dataset into a prompt for large language models (LLMs) can result in excessively long prompts, often exceeding the token limits. This overload can dilute the context, making it difficult for LLMs to focus on relevant details, which can degrade error correction (EC) performance and increase computational costs unnecessarily.

In ZeroEC, the *contextual-relevant tuple search* (CTS) module designs a novel *contextual-relevant tuple* (CT) retriever that selects the most contextually correlated tuples for each dirty tuple, rather than using the entire dataset. This retriever can significantly shorten lengthy prompts and reduce computational costs, while simultaneously preserving the complementary contexts necessary for efficient and effective EC. In this section, we first explain how to select correlative columns for data retrieval. We then detail the construction of the CT retriever to acquire the contextual information required for EC.

A. Correlative Column Selection

Incorporating irrelevant columns into the retrieved tuples introduces noisy information and inflates the prompt [21]. In the CTS module, we introduce a *mutual information based column selection* (MS) strategy that employs *mutual information* (MI) [33] to evaluate inter-column correlations and filter out irrelevant columns for efficient data retrieval.

Specifically, the MS strategy employs the MI metric to evaluate inter-column correlations by quantifying the reduction

in uncertainty about the target dirty column given the value of a specific column in the relational dataset. Columns with higher MI values relative to the target dirty column tend to contain more complementary information, which is crucial for effective EC. The MI value between the column $\mathbf{M} \in \mathcal{A}$ and the target dirty column $\mathbf{N} \in \mathcal{A}$ can be defined as follows:

$$E_m(\mathbf{M}; \mathbf{N}) = \sum_{m \in \mathbf{M}} \sum_{n \in \mathbf{N}} p(m, n) \cdot \log \left(\frac{p(m, n)}{p(m) \cdot p(n)} \right),$$

where $p(m, n)$ represents the joint probability distribution of the columns $\mathbf{M}$ and $\mathbf{N}$. $p(\mathbf{M})$ and $p(\mathbf{N})$ denote the marginal probability distributions of $\mathbf{M}$ and $\mathbf{N}$, respectively. Then, the MI values of all column pairs are normalized, i.e., the normalized MI $\hat{E}_m(\mathbf{M}; \mathbf{N})$ for $(\mathbf{M}; \mathbf{N})$ is calculated by:

$$\hat{E}_m(\mathbf{M}; \mathbf{N}) = \frac{E_m(\mathbf{M}; \mathbf{N})}{H(\mathbf{N})} = \frac{E_m(\mathbf{M}; \mathbf{N})}{-\sum_{n \in \mathbf{N}} p(n) \cdot \log(p(n))},$$

where $H(\mathbf{N})$ is the Shannon entropy [33] of the column $\mathbf{N}$.

Using this normalized MI metric, the MF strategy selects a column $\mathbf{a}_i \in \mathcal{A}$ for each dirty one $\mathbf{a}_t$, if $\hat{E}_m(\mathbf{a}_i; \mathbf{a}_t)$ exceeds a threshold α , forming the correlative column set $\mathbf{S}_t$, i.e.,

$$\mathbf{S}_t = \{\mathbf{a}_i \mid \hat{E}_m(\mathbf{a}_i; \mathbf{a}_t) \geq \alpha\}.$$

In this paper, we set α to 0.5 to balance the coverage of relevant columns with the need for high informativeness, ensuring the retention of key features while minimizing information loss [34]. This strategy ensures that the most relevant columns, with higher MI values, are selected for constructing CT retrievers.

B. Contextual-relevant Tuple Retriever

On top of the correlative columns, we develop a *contextual-relevant tuple* (CT) retriever to efficiently and effectively choose tuples that are most contextually related to dirty tuples. Specifically, we first develop a *weighted cosine similarity* (WS) function to evaluate the tuple similarity. Then, we build

CT retrievers with the WS function to efficiently retrieve the most relevant tuples for each dirty tuple.

Weighted cosine similarity function. The *weighted cosine similarity* (WS) function is to evaluate the similarity between the target tuple $\mathbf{t}_i$ and any other tuple $\mathbf{t}_j (j \neq i)$, i.e.,

$$Sim(\mathbf{t}_i, \mathbf{t}_j) = \sum_{k=1}^{|\mathcal{A}|} d_{ik} \cdot d_{jk} \cdot \hat{E}_m(\mathbf{a}_k; \mathbf{a}_i) \cdot (z_{ik} \cdot z_{jk}^\top), \quad (1)$$

where d_{ik} is the detection state of the value $t_{ik} \in \mathbf{t}_i$. $\mathbf{a}_t$ is the target dirty column. $\hat{E}_m(\mathbf{a}_k; \mathbf{a}_i)$ is the normalized MI value between $\mathbf{a}_k$ and $\mathbf{a}_i$. $z_{ij} = f(t_{ij})$ is the high-dimensional vector of the cell t_{ij} in the tuple $\mathbf{t}_i \in \mathcal{D}$. To calculate $z_{ij} = f(t_{ij})$, we employ the pre-trained model Sentence-BERT [35] to embed each cell t_{ij} in the tuple $\mathbf{t}_i \in \mathcal{D}$ into a normalized high-dimensional vector space, i.e., $f: \mathbb{R}^{d_c} \mapsto \mathbb{R}^{d_v}$, where d_c is the dimensionality of the cell t_{ij} , and d_v is the dimensionality of the normalized vector.

Contextual-relevant tuple retriever construction. By using the WS function, the CTS module builds a *contextual-relevant tuple* (CT) retriever, utilizing the approximate nearest neighbor (ANN) search library Meta Faiss [32], to efficiently find the most relevant tuples for each dirty tuple.

Specifically, the CT retriever first constructs a Faiss tuple index [32] by concatenating the embeddings of all cells in each tuple, where the concatenated embedding of the tuple $\mathbf{t}_i \in \mathcal{D}$ is represented as $[z_{i1}, \dots, z_{i|\mathcal{A}|}]$. It then uses Faiss's ANN search with the WS function to find the top $p (\ll |\mathcal{D}|)$ relevant tuples $\mathcal{D}_p$ for the dirty tuple $\mathbf{t}_i$. For dirty tuples in $\mathcal{D}_p$ with equal weighted cosine similarity scores, the CT retriever further refines the selection by constructing the most relevant tuple set $\mathcal{R}$. It does this by re-ranking the top $k (\ll p)$ tuples from $\mathcal{D}_p$ based on two criteria: (i) whether the target column is clean, and (ii) prioritizing tuples with more clean values among those sharing the same detection status of the target column. This retriever allows for efficiently acquiring the contextual information required for EC by using LLMs.

Analysis. Taking the entire dataset as input incurs a token consumption of $O(|\mathcal{D}| \cdot |\mathcal{A}|)$, where $|\mathcal{D}|$ and $|\mathcal{A}|$ denote the number of tuples and columns in the dataset, respectively. In contrast, the CTS module reduces the token consumption to $O(k \cdot |\mathcal{S}_t|)$, where k is the number of retrieved tuples, and $|\mathcal{S}_t|$ is the number of selected correlative columns, with $k \ll |\mathcal{D}|$ and $|\mathcal{S}_t| \leq |\mathcal{A}|$. Thus, the CTS module, using the MS strategy and CT retriever, significantly reduces computational costs.

Example 2: In Table II, let “Stateavg” be the target dirty column, and consider $\mathbf{t}_7$ as the identified dirty tuple. The CTS module first calculates the normalized MI for each column relative to the target column “Stateavg”, yielding scores of 0.67 for “City”, 0.75 for “State”, 0.35 for “Drug”, 0.75 for “Measure”, 0.66 for “Weight” and 0.72 for “Drug Dosage”. Consequently, “City”, “State”, “Measure”, “Weight”, and “Drug Dosage” are identified as correlative columns due to their relatively high MI score with “Stateavg”. The module then uses the WS function to construct the CT retriever and retrieves the top three relevant tuples: $\mathbf{t}_5$, $\mathbf{t}_6$, and $\mathbf{t}_8$. Finally,

the retrieved tuples are reranked, prioritizing $\mathbf{t}_6$ and $\mathbf{t}_8$ over $\mathbf{t}_5$. Although $\mathbf{t}_5$ achieves the same similarity score as $\mathbf{t}_6$ due to their shared values in “State” and “Measure”, it is ranked lower because it contains fewer clean values.

V. TEC MODULE

In this section, we first explain how the *training-free explainable correction* (TEC) module automatically generates *chain-of-thoughts* (CoTs) for user-corrected representative tuples. We then detail the process of systematically generating EC rules by using LLMs. Finally, we present how to leverage LLMs to produce explainable error corrections.

A. Automatic Correction CoT Generation

CoT prompting enhances LLMs by structuring their reasoning into intermediate steps, aligning with how humans naturally tackle complex EC problems in real-life scenarios [36]. However, manually producing CoT for EC on specific datasets is labor-intensive and prone to inconsistencies [37].

In the TEC module, we first design a *clustering-based representative tuple sampling* (CS) strategy to efficiently select and manually correct representative tuples with a wide variety of data error types. This strategy can reduce the risk of “hallucination” of LLMs during EC, which is often triggered by unique or non-standard data errors. Following this, we design an *automatic correction CoT generation* (CG) prompt using few-shot prompting to automatically generate CoTs for the EC results of user-corrected representative tuples.

Representative Tuple Sampling. The CS strategy employs the K -means clustering algorithm with a *statistic-augmented data representation* and a *greedy* algorithm to select representative tuples for user corrections.

The CS strategy begins by generating statistic-augmented data representation for each dirty tuple, including both semantic embedding and statistic features. To capture the semantic meaning of each cell value, it utilizes a pre-trained Sentence-BERT model to embed a cell value t_{ij} into a normalized high-dimensional vector $f_{sem}(t_{ij})$. The semantic embedding is then augmented by incorporating three types of *statistic frequency* metrics: *value frequency*, *vicinity frequency*, and *pattern frequency*. To be more specific, the value frequency $f_{val}(t_{ij})$ is calculated as the proportion of times t_{ij} appears in its column $\mathbf{a}_j$, i.e., $f_{val}(t_{ij}) = count(t_{ij})/|\mathbf{a}_j|$, where $count(t_{ij})$ is the number of occurrences of t_{ij} in column $\mathbf{a}_j$. $|\mathbf{a}_j|$ refers to the $\mathbf{a}_j$ cardinality. The vicinity frequency $f_{vi}(t_{ij})$ captures the co-occurrence patterns of t_{ij} with values in other columns, defined as its conditional frequency given each value in other columns: $f_{vi}(t_{ij}) = \{count(t_{ij}|t_{iq})|j \neq q, \forall \mathbf{a}_q \in \mathcal{A}\} / count(t_{iq})$, where $count(t_{ij}|t_{iq})$ denotes how often value t_{iq} determines value t_{ij} . Inspired by [38], pattern frequency considers three generalization levels of t_{ij} , with the frequency defined as: $f_{pat_v}(t_{ij}) = count(pat_v(t_{ij}))/|\mathbf{a}_j|$, where $count(pat_v(t_{ij}))$ denotes its frequency in $\mathbf{a}_j$. Thus, the representation of t_{ij} is defined as: $f(t_{ij}) = f_{sem}(t_{ij}) \oplus f_{val}(t_{ij}) \oplus f_{vi}(t_{ij}) \oplus f_{pat}(t_{ij})$. In this way, for a tuple $\mathbf{t}_i$, its representation is the concatenation of the representations of all

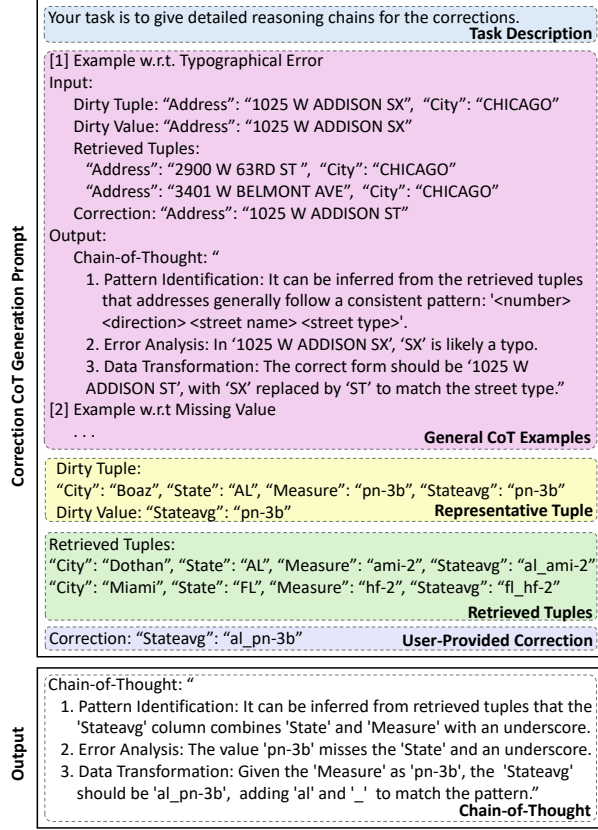


Fig. 2: Correction CoT generation prompt framework

its cell values: $f(\mathbf{t}_i) = \bigoplus_j f(t_{ij})$. Eventually, the Euclidean distance between two tuples $\mathbf{t}_i$ and $\mathbf{t}_j$ can be defined as: $dist(\mathbf{t}_i, \mathbf{t}_j) = ||f(\mathbf{t}_i) - f(\mathbf{t}_j)||_2$.

The CS strategy then employs the K -means clustering algorithm [39] to divide dirty tuples into θ clusters. Next, the CS strategy employs a greedy algorithm to select one representative tuple from each cluster, prioritizing (i) the inclusion of as many dirty columns as possible, and (ii) the coverage of as many dirty cells as possible. The selected θ representative tuples are then corrected by users.

Correction CoT Generation Prompt. To automatically generate CoTs for user-corrected representative tuples, we design a novel *correction CoT generation* (CG) prompt using few-shot prompting to automatically generate correction CoTs for these tuples. Figure 2 provides an example of the CG prompt on the hospital dataset as shown in Table II. This prompt mainly contains the following five components:

- (1) *CG Task Description*: This introduces the correction CoT generation task, i.e., generating reasoning CoTs for EC.
- (2) *General CoT Examples*: Consistent with previous studies [4], [5], [31], we generate four examples by injecting four commonly encountered types of errors [4], [5], [7], [40]—missing values, typographical errors, formatting issues, and violated attribute dependencies—into four random tuples, one for each error type, from the *Chicago*

Food Inspection benchmark dataset [11]. In each example, we use the dirty tuple, dirty value, retrieved tuples, and correction results as inputs, with the output being CoTs. These examples act as general in-context demonstrations, guiding LLMs to automatically generate CoTs.

- (3) *Representative Tuple*: This provides the details of the representative tuple.
- (4) *Retrieved Tuples*: This uses the CT retriever to select the most relevant tuples for the representative tuple.
- (5) *User-provided Correction*: This provides the EC results produced by users for the representative tuple, i.e., correcting "pn-3b" to "al_pn-3b" in the "Stateavg" attribute.

Correction CoT Generation Output. The CG prompt enables LLMs to automatically produce reasoning CoTs for user-corrected representative tuples, e.g., the output in Figure 2. These CoTs are then used to extract EC rules for efficient correction or to create dataset-specific in-context demonstrations, guiding LLMs to perform explainable EC on dirty tuples. Moreover, users can also opt to manually write correction CoTs for a more hands-on approach if desired.

B. Correction Rule Generation

Directly deploying LLMs to correct each erroneous value in large-scale dirty datasets leads to a high volume of LLM invocations, making LLM-based EC systems impractical due to their high computational cost, inefficiency, and time consumption [3]–[5], [26]. Notably, many of these errors can be effectively and efficiently resolved using correction rules [11], [41], such as data patterns and functional dependencies. However, defining these correction rules [3], [11], [15], [16] for specific dirty datasets requires deep expertise in both the datasets and the EC system [3], [18], a level of proficiency that most users do not possess.

To this end, in the TEC module, we devise a *high-quality correction rule generation* (RG) strategy. This strategy aims to automatically generate a set of high-quality EC rules by using a *correction rule generation* (RG) prompt and a *EC rule validation* (RV) mechanism. By employing these generated EC rules to directly correct each dirty record, rather than invoking LLMs for each dirty record, this strategy significantly reduces the frequency of LLM usage and enhances the overall efficiency of EC.

Correction Rule Generation Prompt. For each dirty column, we introduce an *error correction rule generation* (RG) prompt that enables LLMs to autonomously generate diverse types of correction rules, such as data patterns and functional dependencies, for efficient and explainable EC. The RG prompt takes the following two components as inputs:

- (1) *RG Task Description*: This establishes the objective of the RG prompt: to extract EC rules for a target dirty column, adhering to a defined structured output format, based on correction CoTs. For example, to generate a functional dependency for the column "Stateavg", this description specifies that the desired output should include one or more attribute names (separated by commas) that collectively determine the value of "Stateavg".

Algorithm 1: The RV Mechanism

Input: target dirty column N , representative tuples $\mathcal{D}_{\text{err}}^{\text{rep}}$, user-corrected representative tuples $\hat{\mathcal{D}}_{\text{err}}^{\text{rep}}$, clean tuples $\mathcal{D}_{\text{clean}}$, and rule description set Φ

Output: A validated EC rule set Σ_N for N

```
1:  $\Sigma_N \leftarrow \emptyset$  and  $\mathcal{T} \leftarrow \text{Select}(\mathcal{D}_{\text{err}}^{\text{rep}})$ 
2: for  $\varphi \in \Phi$  do
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $\text{Des}_i^\varphi \leftarrow \text{Rephraser}(\varphi, i)$ 
5:      $\text{RG}_i^\varphi \leftarrow \text{Des}_i^\varphi + \text{Work}(\mathcal{T}, N)$ 
6:      $\delta_i^\varphi \leftarrow \text{LLM}(\text{RG}_i^\varphi)$ 
7:     if  $\delta_i^\varphi(\mathcal{D}_{\text{err}}^{\text{rep}}, N) = \hat{\mathcal{D}}_{\text{err}}^{\text{rep}}$  and  $\delta_i^\varphi(\mathcal{D}_{\text{clean}}, N) = \mathcal{D}_{\text{clean}}$ 
       then  $\Sigma_N \leftarrow \Sigma_N \cup \{\delta_i^\varphi\}$ 
8: return  $\Sigma_N$ 
```

- (2) *Representative Tuple Correction Workflow:* This outlines the correction processes for the provided representative tuples. It includes the representative tuple, original dirty value, retrieved tuples, EC result, and reasoning correction CoT. For example, in Table II, t_1 is a representative tuple with the dirty value “pn-3b”, while t_4 and t_8 are the retrieved tuples. The generated reasoning correction CoT is the output in Figure 2.

Correction Rule Generation Output. The TEC module utilizes the RG prompt to interact with LLMs, enabling the automatic generation of various types of explainable EC rules. This results in the formulation of correction rules that can be applied to effectively and efficiently handle errors in the relational data. For example, the LLMs’ outputs may illustrate one functional dependency for correcting the dirty column “Stateavg”, e.g., the columns “State” and “Measure” determine “Stateavg”, expressed as $(\text{State}, \text{Measure}) \rightarrow \text{Stateavg}$.

Generated Rule Validation Mechanism. Due to the stochastic nature of LLMs’ decoding during complex reasoning tasks [42], inaccuracies may arise in the generated EC rules [27], [43]. To deal with this issue, rather than directly applying all generated rules for correction, we develop a *generated rule validation* (RV) mechanism to identify and select robust EC rules from a set of rules generated by LLMs using the RG prompt. Algorithm 1 describes the procedure of the RV mechanism in the ZeroEC model for generating and validating correction rules for the target dirty column N .

Specifically, the RV mechanism begins by randomly selecting half of the representative tuples from $\mathcal{D}_{\text{err}}^{\text{rep}}$ to form the training set $\mathcal{T}$ (line 1). For each rule type φ in the EC rule description set Φ , we employ a prompt rephraser (i.e., Rephraser), which leverages LLMs to reformulate a given instruction, to create n different versions of RG task description in the RG prompt, that could lead to potentially different rule candidates (line 4). Then, with the i -th generated RG task description (i.e., Des_i^φ) and the representative tuple correction workflows (i.e., Work), we employ the RG prompt (i.e., RG_i^φ) to generate the correction rule δ_i^φ w.r.t. φ (lines

5-6) using LLMs. Finally, the output validated EC rule set Σ_N for N consists of generated rules δ_i^φ that correctly rectify all dirty values in N of representative tuples $\mathcal{D}_{\text{err}}^{\text{rep}}$ without corrupting any clean value in N of $\mathcal{D}_{\text{clean}}$ (lines 7-8).

C. Explainable Error Correction

With the support of the generated correction processes of representative tuples and verified high-quality EC rules, we design a *hybrid explainable error correction* (HC) strategy to generate explainable EC correction results for all dirty values. This strategy first designs a *parallel rule execution* (RE) mechanism to efficiently execute multiple rules simultaneously to identify the most appropriate correction for dirty values. However, some errors cannot be resolved by the *parallel rule execution* (RE) mechanism alone, as they involve cases where rules are unavailable, ineffective, or inconclusive. Then, it introduces an *explainable error correction* (ER) prompt to generate both correction results and correction CoTs for the remaining dirty values.

Parallel Rule Execution Mechanism. Using the generated high-quality EC rules, we introduce a *parallel rule execution* (RE) mechanism that efficiently applies multiple rules in parallel to determine the most suitable correction for dirty values. Specifically, for the target dirty column N , the RE mechanism first applies all generated rules in Σ_N in parallel, producing $|\Sigma_N|$ explainable corrections for each dirty value. It then selects the correction supported by the majority as the preferred choice for each dirty value. However, the RE mechanism cannot resolve certain errors, particularly those arising in scenarios where rules are unavailable, ineffective, or yield inconclusive outcomes. Specifically, these cases include: (i) erroneous values in dirty columns without any applicable EC rules, (ii) values that remain uncorrected even after applying the generated rules, and (iii) instances where the rule-based corrections result in a voting tie. To address these cases, LLMs are invoked through the ER prompt for further processing.

Example 3: To demonstrate the application of generated EC rules, consider two types, i.e., *functional dependency* and *data pattern*, generated for the dirty column “Stateavg” in Table II. For the dirty value “hx-2” in t_7 , the functional dependency $[(\text{State}, \text{Measure}) \rightarrow \text{Stateavg}]$ suggests the correction “fl_hf-2”, whereas the functional dependency $[(\text{City}, \text{Measure}) \rightarrow \text{Stateavg}]$ fails to produce a correction due to the dirtiness of the “City” attribute in t_7 . Additionally, the data pattern $[\text{Stateavg} = \text{lowercase}(t[\text{State}]) + \text{'_'} + t[\text{Measure}]]$ also suggests the correction “fl_hf-2”. After applying majority voting, the final EC result for “hx-2” is determined to be “fl_hf-2”.

Explainable Error Correction Prompt. For dirty values that cannot be addressed by the generated EC rules, we design an *explainable error correction* (ER) prompt. This prompt enables LLMs to generate both correction results and explanatory CoTs for the corrections. It takes the following five components as inputs:

- (1) *ER Task Description:* This introduces the task of generating correction results and their detailed reasoning CoTs.

- (2) *General EC Examples*: Similarly, we construct four general EC examples using four user-injected dirty tuples in the *Chicago Food Inspection* dataset. Each example takes the dirty tuple, dirty value, and retrieved tuples as inputs, and outputs the EC results with the reasoning CoTs.
- (3) *Representative Examples*: For each representative tuple containing the target dirty column, we construct a representative example that uses the representative tuple, dirty value, and retrieved tuples as inputs to generate EC results and reasoning CoTs generated using the CG prompt.
- (4) *Dirty Tuple*: It presents the dirty tuple with its corresponding dirty value in the target dirty column.
- (5) *Retrieved Tuples*: It utilizes the CT retriever to select the k tuples most similar to the dirty tuple.

Correction Output. Using the ER prompt, the TEC module leverages LLMs to generate *correction results* and corresponding *correction CoTs* for all dirty tuples. In particular, the correction CoTs detail the reasoning process, outlining the logical steps, considerations, and contextual information during EC. This ensures that the corrections are interpretable, allowing users to understand the LLM’s decision-making process for critical applications.

VI. EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of our system ZeroEC and four state-of-the-art error correction (EC) methods. All methods were implemented in Python and executed on a machine running Ubuntu 20.04 LTS, equipped with an Intel(R) Xeon(R) Platinum 8336C CPU @ 2.30GHz, two NVIDIA RTX 2080 Ti GPUs, and 512 GB of RAM.

Datasets. In the experiments, we use six real-world datasets and one simulated private dataset. Table III lists the information of each dataset, including the number of samples (#Tuples), attributes (#Attrs), error rate, and error types, respectively. In particular, *Hospital* and *Flights* datasets [11] include a high degree of duplicate tuples and correlated columns. *Beers* [4] is a real-world dataset sourced through web scraping and manually cleaned by the dataset owner. *Rayyan* [5] is another real-world dataset cleaned by its owners. It contains all the error types, thus making it hard to repair. *Tax* [40] is a large dataset describing tax payment records. It contains various data error types with 200,000 tuples and 15 attributes. The *IMDB*² dataset is a substantial collection of millions of movies and TV series from 1905 to 2022, containing complex error types challenging to fix. The *Environment* dataset is a synthetic and private dataset. It includes samples with environmental features such as climate zones, PM 2.5 levels, and temperature, and encompasses all typical data error types.

Four kinds of error types, namely *missing value* (MV), *typo* (T), *violated attribute dependency* (VAD), and *formatting issue* (FI), exist in the datasets, and are primarily encountered in real-world scenarios [4], [5], [7], [40].

Baselines. In the experiments, the baselines include state-of-the-art EC methods, i.e., Scare [10], HoloClean [11], Baran

TABLE III: Datasets for our experiments

Name	#Tuples	# Attrs	Error rate	Error types
<i>Hospital</i>	1,000	20	3%	T, VAD
<i>Flights</i>	2,376	7	30%	MV, FI, VAD
<i>Beers</i>	2,410	11	16%	MV, FI, VAD
<i>Rayyan</i>	1,000	11	9%	MV, T, FI, VAD
<i>Tax</i>	200,000	15	4%	T, FI, VAD
<i>IMDB</i>	1,000,000	6	1%	T, FI, VAD
<i>Environment</i>	1,000	11	11%	MV, T, FI, VAD

[4], and FMs [21]. In particular, *Scare* [10] partitions the dataset and uses the clean values to choose corrections of data errors based on their statistical likelihood. *HoloClean* [11] leverages integrity rules, matching dependencies, and statistical signals to fix data errors holistically. Baran [4] generates candidates based on contexts like vicinity, domain, and string variation, then trains a classifier on manually cleaned data and generated candidates to select the best repair options for other error cells. FMs [21] directly applies LLMs to the EC task without a carefully designed framework.

Metrics. In the evaluation, we employ *precision* (P), *recall* (R), and *F1-score* (F1) to measure the effectiveness of EC models. Higher accuracy values (i.e., P, R, and F1) indicate better correction performance. We also measure the *execution time* (Time) of these methods. A shorter execution time means greater efficiency. Each value is reported as the average of results from five different random data divisions. **Implementation Details.** For *Scare*, the maximum number of corrections is set to the number of detected errors to accommodate all data errors. For *HoloClean* which requires denial constraints (DCs) for input, we mine the DCs on the ground truth datasets using the Hydra algorithm [44]. For *Baran* and *ZeroEC* which require user annotations, the representative tuple size θ is set to 10. In FMs, we employ GPT-3.5-turbo for executing error correction. In *ZeroEC*, we employ GPT-4o for correction CoT and rule generation, and GPT-3.5-turbo for explainable EC. For parameters of LLMs, we set the temperature to 0 to ensure stable and reproducible outputs, and keep other parameters at their default values. The hyper-parameters p , k , and n are set to 30, 3, and 3, respectively. Two widely adopted rules, i.e., functional dependencies and data patterns, are selected as the generated EC rules.

A. Scalability Evaluation

Table IV reports the correction accuracy of EC methods over six public real-world datasets and one simulated private dataset. Some values are unavailable (denoted by “—”), as the corresponding methods cannot finish within 10^5 seconds.

One can observe that, *ZeroEC* significantly outperforms all of the baselines. In terms of prediction accuracy (i.e., P, R, and F1), *ZeroEC* exceeds the best-performing baseline (i.e., Baran) by an average of 66.82%, and even increases up to 313.64% on the *IMDB* dataset w.r.t. R. It is attributed to the effective LLM with the representative tuple sampling and explainable correction. In particular, almost all EC methods

²<https://www.imdb.com/interfaces>

TABLE IV: Error correction performance in comparison to the baselines

System	Hospital			Flights			Beers			Rayyan			Tax			IMDB			Environment		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Scare	0.67	0.53	0.59	0.57	0.06	0.11	0.16	0.07	0.10	0.00	0.00	0.00	—	—	—	—	—	—	0.00	0.00	0.00
Holoclean	0.99	0.51	0.68	0.81	0.38	0.52	0.36	0.09	0.15	0.00	0.00	0.00	—	—	—	—	—	—	0.00	0.00	0.00
Baran	0.86	0.78	0.81	1.00	0.98	0.99	0.85	0.74	0.79	0.54	0.24	0.33	0.91	0.72	0.81	0.53	0.22	0.31	0.73	0.38	0.50
FMs	0.56	0.56	0.56	0.01	0.01	0.01	0.24	0.24	0.24	0.00	0.00	0.00	0.05	0.05	0.05	0.75	0.75	0.75	0.40	0.38	0.39
ZeroEC	0.96	0.96	0.96	0.99	0.99	0.99	0.99	0.99	0.99	0.81	0.80	0.81	0.99	0.99	0.99	0.91	0.91	0.91	0.83	0.83	0.83

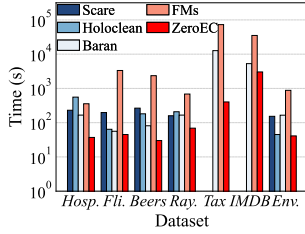


Fig. 3: Time cost of ECs

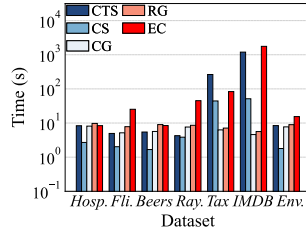
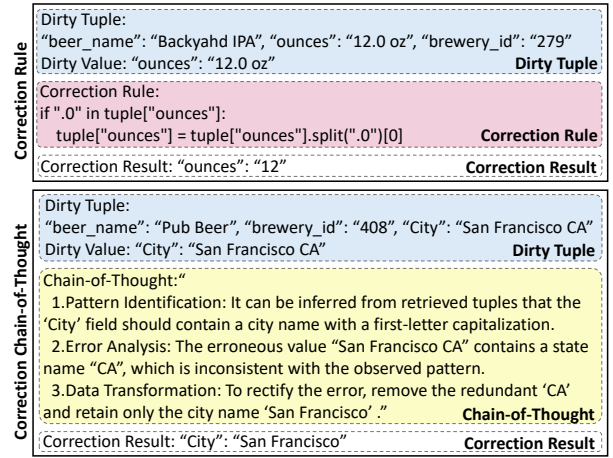


Fig. 4: Time cost in ZeroEC

perform worse on the *Rayyan* dataset compared to other datasets. This is because the *Rayyan* dataset, with its complex and diverse error patterns, poses significant challenges for all EC methods. On this dataset, **ZeroEC** still outperforms other methods. It surpasses the best baseline, *Baran*, by an average of 1.33 times, with an increase of up to 2.33 times in R. In particular, compared to EC model *FMs* which employs the same LLM, **ZeroEC** achieves a significantly greater improvement in correction accuracy. This is due to **ZeroEC**'s CS strategy, which ensures LLMs are exposed to a broad spectrum of data error types and their corresponding manual corrections. This exposure helps minimize the risk of "hallucination" during LLM-generated corrections.

Figure 3 depicts the time consumption of EC methods over seven datasets, i.e., *Hospital* (*Hosp.*), *Flights* (*Fli.*), *Beers*, *Tax*, *IMDB* and *Environment* (*Env.*). The total time consumption includes the training time and testing time. Smaller time consumption indicates better efficiency. In Figure 3, the results of *Scare* and *Holoclean* are unavailable over *Tax* and *IMDB*, since they are not able to finish within 10^5 seconds. One can observe that, **ZeroEC** takes much less time consumption than baselines in most cases. **ZeroEC** is 6.87 times faster than the most efficient baseline *Baran* on average, with a speed increase of up to 31.49 times on the *Tax* dataset. This is because **ZeroEC** leverages LLMs to automatically generate EC rules and directly correct erroneous data, eliminating the need for model training and enabling efficient error correction. Additionally, compared to the LLM-based method *FMs*, **ZeroEC** typically requires only 5.28% of the running time on average, and as little as 0.52% on the *Tax* dataset. This demonstrates the superior efficiency of **ZeroEC**, even on datasets with millions of tuples in real-world scenarios. Due to the high complexity, we omit *Scare* in the rest of the experiments.

Figure 4 shows the time cost for five consecutive steps in **ZeroEC** across six datasets: *contextual-relevant tuples search* (CTS), *clustering-based representative tuples sampling* (CS),

Fig. 5: Interpretability illustration with the *Beers* dataset

correction CoT generation (CG), *correction rule generation* (RG), and *explainable error correction* (EC). It can be observed that the CTS and EC steps contribute the most to the overall time consumption in **ZeroEC** in most cases. On average, the CTS step accounts for 26.08% of the total time, while the EC step contributes 40.39%. In addition, the CG and RG steps exhibit lower and relatively stable time consumption. This is because these two steps generate correction CoTs and correction rules only for the representative tuples, whose size remains consistent across all seven datasets.

Moreover, Figure 5 illustrates a generated correction rule and a correction CoT of **ZeroEC** for two random samples from the *Beers* dataset. As shown, **ZeroEC** provides clear and interpretable correction strategies, allowing for a better understanding of how errors are addressed. These explanations facilitate tracing the underlying error correction logic, enhancing **ZeroEC**'s utility for practical EC applications.

B. Parameter Evaluation

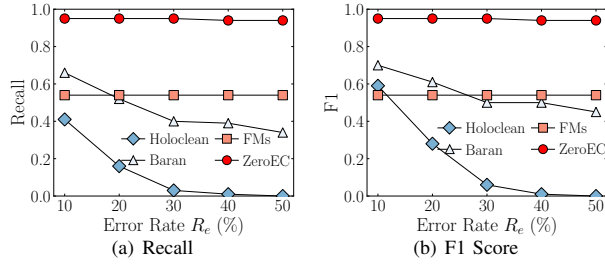
Effect of error rate. When varying the error rate R_e from 10% to 50%, the corresponding results of EC methods on the *Hospital* dataset are depicted in Figure 6. We can find that, with the growth of the error rate, each algorithm's correction accuracy (i.e., R, and F1) descends consistently. The reason is that, the available data information turns less with the increase of R_e , resulting in a lower correction accuracy. Among these algorithms, **ZeroEC** performs best in each case. Its accuracy stabilizes as the error rate increases. In other words, **ZeroEC** is

TABLE V: Maximum calculation resource usage of the algorithms across varying data sizes

System	Tax-10k			Tax-20k			Tax-30k			Tax-40k			Tax-50k		
	Time	CPU	Memory	Time	CPU	Memory	Time	CPU	Memory	Time	CPU	Memory	Time	CPU	Memory
Holoclean	1,872	1203%	191,019	—	—	—	—	—	—	—	—	—	—	—	—
Baran	273	1299%	2,284	510	1362%	4,874	952	1567%	8,499	1,523	1450%	13,066	2,037	1511%	18,513
FMs	4,244	106%	532	8,125	107%	594	11,824	107%	645	15,281	108%	696	18,192	107%	758
ZeroEC	61	1199%	2,509	69	1201%	3,604	87	1199%	4,884	113	1203%	5,581	126	1199%	6,737

TABLE VI: Performance with different LLMs

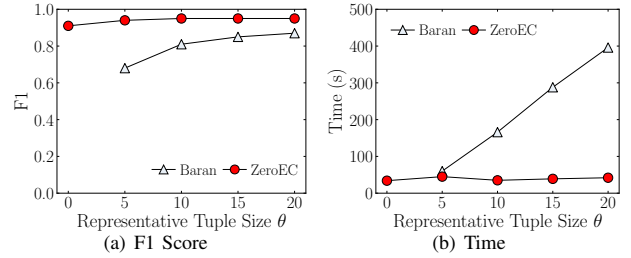
LLM	Hospital			Flights			Beers			Rayyan			Tax			IMDB		
	F1	Time	Cost	F1	Time	Cost	F1	Time	Cost	F1	Time	Cost	F1	Time	Cost	F1	Time	Cost
GPT-3.5-turbo	0.96	37	0.17	0.99	45	0.80	0.99	30	0.01	0.81	69	1.96	0.99	402	4.29	0.91	3,021	58.64
Claude-3.5-haiku	0.96	48	0.14	0.99	60	0.53	0.99	37	0.01	0.55	155	1.21	0.99	545	3.77	0.90	3,564	38.12
Qwen-2.5-72b	0.96	47	0.19	0.99	666	0.97	0.99	43	0.01	0.76	177	2.25	0.99	593	5.01	0.90	4,960	69.78
GPT-4o	0.97	54	0.87	1.00	81	4.18	0.99	43	<u>0.06</u>	0.87	211	11.13	0.99	669	27.28	0.91	7,577	288.51

Fig. 6: The performance of ZeroEC vs. R_e

more robust with the increasing error rate R_e than others. The reason behind this is that ZeroEC exposes LLMs to a wide variety of data error types and their corresponding manual corrections, to decrease or delay the impact of increased error rate on the correction performance.

Effect of sample size. Table V presents the execution time (in seconds), maximum CPU usage (in percent), and maximum memory usage (in MB) of the EC methods on the *Tax* dataset, with sample sizes ranging from 10,000 to 50,000 records (i.e., *Tax*-10k to *Tax*-50k). Notably, Holoclean exhibits exceptionally high memory consumption, utilizing up to 191,019 MB on the *Tax*-10k dataset, and is incapable of handling larger datasets due to its excessive memory requirements. ZeroEC requires significantly less model execution time than others, achieving an average speedup of 10.70 times compared to the most efficient baseline, Baran. Meanwhile, ZeroEC also demonstrates lower memory consumption, using only 64% of the memory required by Baran on average. This efficiency is achieved by eliminating the need for model training and avoiding the storage of model parameters. While FMs exhibits low memory usage due to its simple implementation, it is significantly slower than ZeroEC, with an average time consumption 120.33 times higher. The reason is that ZeroEC generates EC rules to directly correct the majority of dirty records, eliminating the need to invoke LLMs individually for each record.

Effect of representative tuple size. With the representative

Fig. 7: The performance of ZeroEC and Baran vs. θ

tuple size θ varying from 0 to 20, Figure 7 plots the corresponding results (i.e., F1 and Time) of Baran and ZeroEC over the *Hospital* dataset. It is important to note that Baran cannot train the EC model when θ is 0, as it depends on user-labeled tuples for model training. In contrast, ZeroEC can still effectively perform EC under the same condition by learning from general EC examples. We can find that ZeroEC outperforms all baselines, achieving higher correction accuracy and lower time consumption. Moreover, with the growth of θ , the correction accuracy of each algorithm consistently improves. The reason is that, a larger θ provides more labeled samples for the EC models, enhancing their effectiveness. ZeroEC maintains more stable and lower time consumption than Baran as θ increases. These results further validate the effectiveness and efficiency of ZeroEC.

Effect of LLMs. Table VI depicts the experimental results, i.e., F1, execution time (in seconds), and money cost (in \$), of ZeroEC with different kinds of advanced LLMs, i.e., GPT-3.5-turbo³, Claude-3.5-haiku⁴, Qwen-2.5-72b⁵, and GPT-4o⁶. We can find that, ZeroEC with GPT-3.5-turbo reduces execution time to 70.87%, 64.00%, and 54.34% compared to Claude-3.5-haiku, Qwen-2.5-72b, and GPT-4o, respectively. It achieves a correction accuracy increase of 8.06% and 1.28% relative to Claude-3.5-haiku and Qwen-2.5-72b, with a minor

³<https://platform.openai.com/docs/models#gpt-3-5-turbo>

⁴<https://www.anthropic.com/claude/haiku>

⁵<https://www.aliyun.com/product/bailian>

⁶<https://platform.openai.com/docs/models#gpt-4o>

TABLE VII: The ablation study of ZeroEC

System	Hospital				Flights				Beers				Rayyan				Tax			
	P	R	F1	Time	P	R	F1	Time	P	R	F1	Time	P	R	F1	Time	P	R	F1	Time
ZeroEC w/o MS	0.88	0.88	0.88	38	0.99	0.99	0.99	44	0.99	0.99	0.99	34	0.54	0.46	0.50	72	0.99	0.99	0.99	349
ZeroEC w/o CTS	0.62	0.61	0.61	32	0.23	0.20	0.21	243	0.99	0.99	0.99	29	0.77	0.69	0.73	64	0.99	0.99	0.99	137
ZeroEC w/o CS	0.89	0.89	0.89	56	0.94	0.94	0.94	43	0.96	0.96	0.96	30	0.49	0.44	0.46	64	0.99	0.98	0.98	382
ZeroEC w/o CG	0.91	0.91	0.91	30	0.85	0.85	0.85	42	0.67	0.66	0.67	26	0.10	0.08	0.09	61	0.99	0.99	0.99	385
ZeroEC w/o RG	0.95	0.95	0.95	57	0.98	0.98	0.98	257	0.99	0.99	0.99	243	0.81	0.70	0.75	58	0.99	0.98	0.99	5,541
ZeroEC w/o ER	1.00	0.70	0.82	30	1.00	0.94	0.97	20	1.00	0.99	0.99	25	0.00	0.00	0.00	23	1.00	0.98	0.99	296
ZeroEC	0.96	0.96	0.96	37	0.99	0.99	0.99	45	0.99	0.99	0.99	30	0.81	0.80	0.81	69	0.99	0.99	0.99	402

TABLE VIII: Correction performance with imperfect detection

ED	System	Hospital	Flights	Beers	Rayyan	Tax
Raha	Holoclean	0.50	0.49	0.15	0.00	—
	Baran	0.56	0.51	0.76	0.23	0.80
	FMs	0.40	0.00	0.22	0.00	0.05
	ZeroEC	0.66	0.51	0.96	0.67	0.87
Raha	Perfect EC	0.69	0.89	0.99	0.85	0.92
Perfect ED	ZeroEC	0.96	0.99	0.99	0.81	0.99

0.34% accuracy trade-off compared to GPT-4o. Moreover, GPT-3.5-turbo reduces monetary expenses by 11.88% and 81.82% compared to Qwen-2.5-72b and GPT-4o, respectively, while incurring a 33.66% increase in cost compared to Claude-3.5-haiku. Thus, if users want to maximize the efficiency of ZeroEC, they can choose GPT-3.5-turbo. If they would like to maximize the effectiveness of ZeroEC, the advanced system GPT-4o can be chosen. Claude-3.5-haiku stands out as an economical choice for cost-conscious users. Additionally, in scenarios where data privacy is a key concern, instead of using online APIs, deploying the open-source Qwen-2.5-72b model on private servers provides a practical and secure solution.

C. Ablation Study

We investigate the influence of different elements of ZeroEC on the EC performance. The corresponding experimental results over the first five datasets, including P, R, F1, and Time (in seconds), are presented in Table VII. ZeroEC w/o MS (and ZeroEC w/o CG) is the variant of ZeroEC that excludes the MS strategy (and the CG prompt). ZeroEC w/o CS is the variant of ZeroEC that replaces the CS strategy with a simple *random sampling* method. ZeroEC w/o CTS is the variant of ZeroEC without the CTS module. To avoid exceeding the LLM’s maximum prompt length, this variant uses no tuple as input instead of the entire dataset. ZeroEC w/o RG (and ZeroEC w/o ER) is the variant of ZeroEC without the RG strategy (and the ER prompt).

We observe that each of the six components in ZeroEC—the MS strategy, CTS module, CS strategy, CG prompt, RG strategy, and ER prompt—contribute positively to correction performance. Particularly, the MS strategy filters out irrelevant columns in the retrieved tuples and proves essential for datasets with numerous columns (e.g., in *Hospital*) or many long strings (e.g., in *Rayyan*). Its removal results in

TABLE IX: Results of downstream performance

ED	EC	Classification		Regression ($\times 10^{-5}$)	
		P	R	MSE	MAE
Raha	Holoclean	0.26	0.26	19.35	672.11
	Baran	0.25	0.27	9.30	381.54
	FMs	0.24	0.27	9.03	346.90
	ZeroEC	0.30	0.32	8.44	341.33
Dirty Data		0.23	0.25	19.23	667.51
Clean Data		0.34	0.32	8.44	341.33

performance decreases of 6.60% and 37.21%, respectively. The CTS module retrieves contextually relevant tuples for dirty values, and its removal leads to a significant performance drop, with a 36.11% and 78.45% decline on datasets like *Hospital* and *Flights*, which rely on related contexts for correction. The CS strategy samples representative tuples for user labeling, and its removal has a profound impact on datasets with diverse error types, such as *Rayyan*, resulting in a 42.57% decrease in performance. The removal of the CG prompt significantly impacts datasets with complex error patterns, such as *Beers* and *Rayyan*, where multi-step reasoning is necessary, causing performance to drop by 32.66% and 88.85%, respectively. The RG strategy is essential for generating correction rules, and its absence leads to a drastic decrease in efficiency, with a 92.74% decline on the *Tax* dataset which contains a high volume of errors. Finally, the ER prompt leverages LLMs for correction, and its removal results in a complete loss of accuracy, with a 100% decline on datasets like *Rayyan*, where no EC rules can be applied. These findings underscore the indispensability of all six components in ZeroEC, demonstrating their collective importance for both effectiveness and efficiency.

D. Correction with Imperfect Error Detection

The effectiveness of EC is inextricably linked to the preceding error detection process. In this part, we consider the entire data cleaning pipeline, acknowledging the imperfections of error detection methods. To evaluate data cleaning performance, we use Raha [5], a state-of-the-art error detection system. Our evaluation comprises three scenarios: (i) perfect error detection (Perfect ED) followed by ZeroEC, (ii) Raha with perfect correction (Perfect EC), representing the upper bound of EC methods, and (iii) Raha with existing correction methods, including HoloClean, Baran, FMs and ZeroEC.

Table VIII reports the experimental results (i.e., F1) of data cleaning pipelines on the first five datasets. We can find that, when detection is imperfect, all correction methods exhibit a decrease in accuracy. This is because undetected errors cannot be addressed by EC models, and falsely detected values may misdirect the generation of correction strategies. However, ZeroEC consistently outperforms all baselines, surpassing the best-performing baseline Baran by an average of 48.85%. In particular, ZeroEC's performance is remarkably close to that of Perfect EC on the *Hospital* and *Beers* datasets, underscoring its robustness in the presence of detection inaccuracies. This resilience highlights ZeroEC's potential as a reliable solution for EC in real-world scenarios, where perfect error detection is often unattainable.

E. Correction Effect on Downstream Tasks

The ultimate goal of correcting error data is to benefit the downstream analytics. We conduct experiments encompassing two common ML tasks, i.e., regression and classification. These experiments compare original dirty data (Dirty Data), ground truth clean data (Clean Data), and data repaired by selected methods.

For the classification task targeting "Beer Style" and the regression task targeting "Alcohol by Volume" in the *Beers* dataset, the prediction results are shown in Table IX. Higher P or R values indicate better prediction performance, while MSE or MAE values are the opposite. In the third scenario, EC methods with Raha are first employed to clean error values in the dirty datasets. Then, a regression/classification model is trained on the cleaned data using the decision tree model. For each experiment, we partition the data into 80% training sets and 20% testing sets.

We can observe that, the prediction performance across different EC algorithms aligns with their correction performance, i.e., ZeroEC consistently outperforming the baselines. Specifically, ZeroEC exceeds the best-performing baseline FMs by 12.90% on average, and it increases up to 25.00% on the classification task w.r.t. P. Moreover, compared to training on the ground truth clean data, ZeroEC shows only a 2.94% decrease in prediction accuracy, while Baran exhibits a more substantial decrease of 16.02%. Conversely, when trained on the original dirty data, ZeroEC demonstrates a 40.85% increase in prediction accuracy, whereas Baran shows a modest 27.79% improvement. These results further confirm the effectiveness of the proposed ZeroEC system.

VII. RELATED WORK

Error correction, EC. EC methods can be classified into two main categories, i.e., *pre-configuration* ones and *configuration-free* ones. In particular, pre-configuration EC methods usually employ integrity rules, e.g., functional dependencies [45], [46], denial constraints [11], [14]–[16] for EC, such as Unified [47], Holistic [15], Relative [46], Nadeef [48], Bigdancing [49], Holoclean [11], Daisy [14], Horizon [50], and MLNClean [16]. However, for most non-expert users, providing rules or constraints is challenging due to the need

for prior knowledge of both the dataset and the EC model. Configuration-free EC methods typically require a few user annotations, and then train machine learning models to generalize the annotations to develop EC strategies for previously unseen data, including Scare [10], BoostClean [13], Baran [4], and GIDCL [24]. While these approaches lessen reliance on expert knowledge, they still require extensive feature engineering or labeled data for intensive model training, making them inefficient and difficult to apply in complex scenarios, like undefined error patterns and intricate textual errors [18].

Large Language Models, LLMs. Recent studies have demonstrated that scaling pre-trained language models (PLMs) in terms of model or data size often improves performance on various downstream tasks, as observed in the scaling laws [51]. Notable examples include the GPT series [27], [52] and open-source LLMs like Qwen-2.5-72b [53]. LLMs, compared to PLMs, exhibit emergent abilities such as in-context learning [54]–[56], instruction following [57]–[59], and step-by-step reasoning [36], [60], enabling them to generate complex outputs with minimal examples. Recent pioneering works focus on transforming data cleaning tasks into generation tasks and applying LLMs to solve them, like error detection [20]–[22], data imputation [20], [22], [25], [26], and error correction [21]. Applying LLMs to the EC task without a carefully designed framework like FMs [21] poses great challenges, including inefficiencies due to token limitations that hinder context capture, susceptibility to generating incorrect or fictional corrections (i.e., "hallucination") [29], [30], and a lack of interpretability necessary for critical sectors like healthcare and government. To our knowledge, there is a significant gap in systematic research on effectively integrating LLMs into an EC framework to address these challenges.

VIII. CONCLUSION

In this paper, we introduce a zero-training and interpretable error correction system ZeroEC with LLMs to efficiently and effectively correct erroneous values. It consists of a CTS module and a TEC module. CTS builds the CT retriever using the WS function to efficiently search the most relevant tuples for dirty tuples. TEC designs the CS strategy that reduces "hallucination" risk by exposing LLMs to diverse data error types. It then develops the CG, RG, and ER prompts for automatically generating rules and explanations for EC. Extensive experiments on real-world datasets demonstrate that, ZeroEC greatly beats the state-of-the-art error correction methods in accuracy and efficiency. In the future, we intend to develop an end-to-end data cleaning system powered by LLMs executing ED and EC iteratively, so as to progressively refine the performance of both processes.

Acknowledgments. This work is supported by the National Key R&D Program under Grant No. 2023YFC2706404, the National NSFC under Grant No. 62372404, the Leading Goose R&D Program of Zhejiang under Grant No. 2024C01109, and the Fundamental Research Funds for the Central Universities under Grant No. 226-2024-00030. Xiaoye Miao is the corresponding author of the work.

REFERENCES

- [1] X. Miao, Y. Wu, L. Chen, Y. Gao, and J. Yin, "An experimental survey of missing data imputation algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 7, pp. 6630–6650, 2022.
- [2] Y. Wu, X. Miao, Z.-a. Nan, J. Zhang, J. He, and J. Yin, "Effective and efficient multi-view imputation with optimal transport," *IEEE Transactions on Knowledge & Data Engineering*, no. 01, pp. 1–12, 2024.
- [3] Z. Abedjan, X. Chu, D. Deng, R. C. Fernandez, I. F. Ilyas, M. Ouzzani, P. Papotti, M. Stonebraker, and N. Tang, "Detecting data errors: Where are we and what needs to be done?" *Proceedings of the VLDB Endowment*, vol. 9, no. 12, p. 993–1004, 2016.
- [4] M. Mahdavi and Z. Abedjan, "Baran: Effective error correction via a unified context representation and transfer learning," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, p. 1948–1961, 2020.
- [5] M. Mahdavi, Z. Abedjan, R. Castro Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang, "Raha: A configuration-free error detection system," in *SIGMOD*, 2019, p. 865–882.
- [6] Y. Wu, J. Wang, X. Miao, W. Wang, and J. Yin, "Differentiable and scalable generative adversarial models for data imputation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 2, pp. 490–503, 2024.
- [7] M. Pham, C. A. Knoblock, M. Chen, B. Vu, and J. Pujara, "Spade: A semi-supervised probabilistic approach for detecting errors in tables," in *IJCAI*, 8 2021, p. 3543–3551.
- [8] A. Heidari, J. McGrath, I. F. Ilyas, and T. Rekatsinas, "Holodetect: Few-shot learning for error detection," in *SIGMOD*, 2019, p. 829–846.
- [9] F. Neutatz, M. Mahdavi, and Z. Abedjan, "Ed2: A case for active learning in error detection," in *CIKM*, 2019, p. 2249–2252.
- [10] M. Yakout, L. Berti-Équille, and A. K. Elmagarmid, "Don't be scared: Use scalable automatic repairing with maximal likelihood and bounded changes," in *SIGMOD*, 2013, p. 553–564.
- [11] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holoclean: Holistic data repairs with probabilistic inference," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, p. 1190–1201, 2017.
- [12] M. K. Hasan and M. Mahdavi, "Automatic error correction using the wikipedia page revision history," in *CIKM*, 2021, p. 3073–3077.
- [13] S. Krishnan, M. J. Franklin, K. Goldberg, and E. Wu, "Boostclean: Automated error detection and repair for machine learning," *ArXiv Preprint ArXiv:1711.01299*, 2017.
- [14] S. Giannakopoulou, M. Karpachiotakis, and A. Ailamaki, "Cleaning denial constraint violations through relaxation," in *SIGMOD*, 2020, p. 805–815.
- [15] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *ICDE*, 2013, pp. 458–469.
- [16] C. Ge, Y. Gao, X. Miao, B. Yao, and H. Wang, "A hybrid data cleaning framework using markov logic networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 5, pp. 2048–2062, 2022.
- [17] H. Mannila and K.-J. Räihä, "Algorithms for inferring functional dependencies from relations," *Data & Knowledge Engineering*, vol. 12, no. 1, pp. 83–99, 1994.
- [18] W. Ni, X. Miao, X. Zhao, Y. Wu, S. Liang, and J. Yin, "Automatic data repair: Are we ready to deploy?" *Proceedings of the VLDB Endowment*, vol. 17, no. 10, p. 2617–2630, aug 2024.
- [19] X. Miao, Y. Wu, L. Chen, Y. Gao, J. Wang, and J. Yin, "Efficient and effective data imputation with influence functions," *Proceedings of the VLDB Endowment*, vol. 15, no. 3, pp. 624–632, 2021.
- [20] P. Li, Y. He, D. Yashar, W. Cui, S. Ge, H. Zhang, D. R. Fainman, D. Zhang, and S. Chaudhuri, "Table-gpt: Table-tuned gpt for diverse table tasks," *ArXiv Preprint ArXiv:2310.09263*, 2023.
- [21] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *ArXiv Preprint ArXiv:2205.09911*, 2022.
- [22] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: A large language model for data preprocessing," *ArXiv Preprint ArXiv:2312.01678*, 2023.
- [23] W. Ni, K. Zhang, X. Miao, X. Zhao, Y. Wu, and J. Yin, "Iterclean: An iterative data cleaning framework with large language models," in *TURC*, 2024, pp. 100–105.
- [24] M. Yan, Y. Wang, Y. Wang, X. Miao, and J. Li, "Gidcl: A graph-enhanced interpretable data cleaning framework with large language models," in *SIGMOD*, 2024, pp. 1–29.
- [25] A. Brinkmann, R. Shraga, and C. Bizer, "Product attribute value extraction using large language models," *ArXiv Preprint ArXiv:2310.12537*, 2023.
- [26] Z. Chen, L. Cao, S. Madden, T. Kraska, Z. Shang, J. Fan, N. Tang, Z. Gu, C. Liu, and M. Cafarella, "Seed: Domain-specific data curation with large language models," *ArXiv Preprint ArXiv:2310.00749*, 2023.
- [27] T. B. Brown, "Language models are few-shot learners," *ArXiv Preprint ArXiv:2005.14165*, 2020.
- [28] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Open and efficient foundation language models," *ArXiv Preprint ArXiv:2302.13971*, vol. 2302, 2023.
- [29] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," *ArXiv Preprint ArXiv:2109.07958*, 2021.
- [30] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ArXiv Preprint ArXiv:2311.05232*, 2023.
- [31] E. Rahm, H. H. Do *et al.*, "Data cleaning: Problems and current approaches," *IEEE Database Engineering Bulletin*, vol. 23, no. 4, pp. 3–13, 2000.
- [32] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [33] C. E. Shannon, "A mathematical theory of communication," *The Bell system technical journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [34] H. Wang, T. M. Khoshgoftaar, and J. Van Hulse, "A comparative study of threshold-based feature selection techniques," in *GrC*, 2010, pp. 499–504.
- [35] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *ArXiv Preprint ArXiv:1908.10084*, 2019.
- [36] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [37] Z. Zhang, A. Zhang, M. Li, and A. Smola, "Automatic chain of thought prompting in large language models," *ArXiv Preprint ArXiv:2210.03493*, 2022.
- [38] Z. Huang and Y. He, "Auto-detect: Data-driven error detection in tables," in *SIGMOD*, 2018, pp. 1377–1392.
- [39] K. Krishna and M. N. Murty, "Genetic k-means algorithm," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 29, no. 3, pp. 433–439, 1999.
- [40] P. C. Arocena, B. Glavic, G. Mecca, R. J. Miller, P. Papotti, and D. Santoro, "Messing up with bart: Error generation for evaluating data-cleaning algorithms," *Proceedings of the VLDB Endowment*, vol. 9, no. 2, p. 36–47, 2015.
- [41] J. Peng, D. Shen, N. Tang, T. Liu, Y. Kou, T. Nie, H. Cui, and G. Yu, "Self-supervised and interpretable data cleaning with sequence generative adversarial networks," *Proceedings of the VLDB Endowment*, vol. 16, no. 3, pp. 433–446, 2022.
- [42] W. S. Saba, "Stochastic llms do not understand language: Towards symbolic, explainable and ontologically based llms," in *ICCM*, 2023, pp. 3–19.
- [43] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *ArXiv Preprint ArXiv:2203.11171*, 2022.
- [44] T. Bleifuß, S. Kruse, and F. Naumann, "Efficient denial constraint discovery with hydra," *Proceedings of the VLDB Endowment*, vol. 11, no. 3, p. 311–323, 2017.
- [45] S. Abiteboul, R. Hull, and V. Vianu, *Foundations of databases*. Addison-Wesley Reading, 1995, vol. 8.
- [46] G. Beskales, I. F. Ilyas, L. Golab, and A. Galiullin, "On the relative trust between inconsistent data and inaccurate constraints," in *ICDE*, 2013, pp. 541–552.
- [47] F. Chiang and R. J. Miller, "A unified model for data and constraint repair," in *ICDE*, 2011, pp. 446–457.
- [48] A. Ebaid, A. Elmagarmid, I. F. Ilyas, M. Ouzzani, J.-A. Quiane-Ruiz, N. Tang, and S. Yin, "Nadeef: A generalized data cleaning system," *Proceedings of the VLDB Endowment*, vol. 6, no. 12, pp. 1218–1221, 2013.
- [49] Z. Khayyat, I. F. Ilyas, A. Jindal, S. Madden, M. Ouzzani, P. Papotti, J.-A. Quiané-Ruiz, N. Tang, and S. Yin, "Bigdancing: A system for big data cleansing," in *SIGMOD*, 2015, p. 1215–1230.

- [50] E. K. Rezig, M. Ouzzani, W. G. Aref, A. K. Elmagarmid, A. R. Mahmood, and M. Stonebraker, "Horizon: Scalable dependency-driven data cleaning," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2546–2554, 2021.
- [51] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *ArXiv Preprint ArXiv:2001.08361*, 2020.
- [52] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *ArXiv Preprint ArXiv:2303.08774*, 2023.
- [53] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang *et al.*, "Qwen2 technical report," *ArXiv Preprint ArXiv:2407.10671*, 2024.
- [54] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, and Z. Sui, "A survey on in-context learning," *ArXiv Preprint ArXiv:2301.00234*, 2022.
- [55] S. Min, M. Lewis, L. Zettlemoyer, and H. Hajishirzi, "Metaicl: Learning to learn in context," *ArXiv Preprint ArXiv:2110.15943*, 2021.
- [56] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," *ArXiv Preprint ArXiv:2111.02080*, 2021.
- [57] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.
- [58] V. Sanh, A. Webson, C. Raffel, S. H. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, T. L. Scao, A. Raja *et al.*, "Multitask prompted training enables zero-shot task generalization," *ArXiv Preprint ArXiv:2110.08207*, 2021.
- [59] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *ArXiv Preprint ArXiv:2109.01652*, 2021.
- [60] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 199–22 213, 2022.